

Deep learning

RNN - Auto-encodeur

Vincent Lefieux



Plan

RNN

Autoencoders

RNN

Autoencoders

Références

Plan

RNN

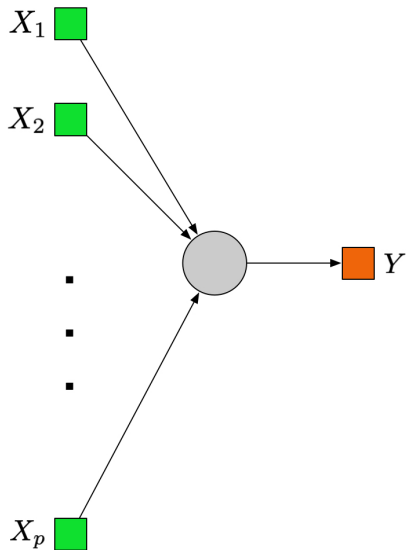
RNN

Autoencoders

Références

- ▶ Les **réseaux de neurones récurrents (RNN)** permettent de traiter des **séquences**, à savoir des :
 - ▶ données liées entre elles,
 - ▶ des données de taille variable.
- ▶ Les RNN sont particulièrement adaptés au traitement de **séries temporelles**, de **textes** et de **sons**.
- ▶ Les neurones d'un DNN ne conservent aucune mémoire contrairement aux **neurones récurrents**.

Neurone classique



RNN

Autoencoders

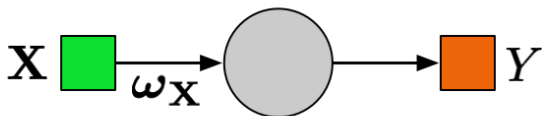
Références

Neurone classique : représentation condensée

RNN

Autoencoders

Références



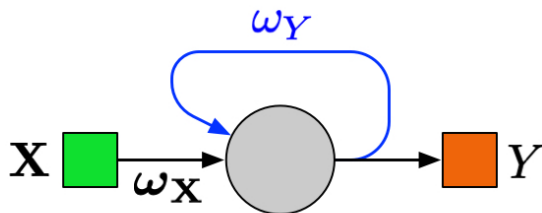
$$Y = \varphi(\omega_0 + \omega_{\mathbf{X}}^{\top} \mathbf{X})$$

Neurone récurrent

RNN

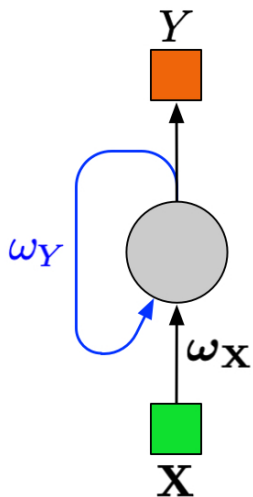
Autoencoders

Références



$$Y_t = \varphi \left(\omega_{\mathbf{X}}^{\top} \mathbf{X}_t + \omega_Y Y_{t-1} + \omega_0 \right)$$

Neurone récurrent : représentation usuelle

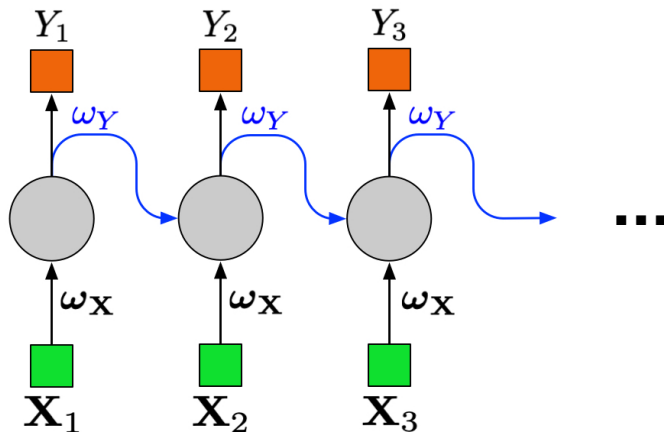


RNN

Autoencoders

Références

Neurone récurrent : version dépliée

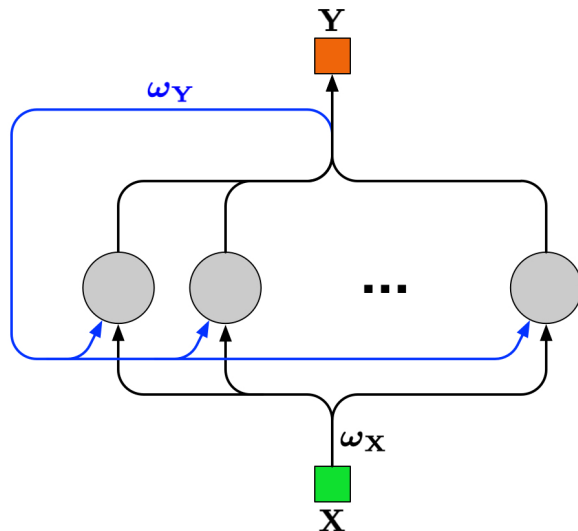


RNN

Autoencoders

Références

Unité récurrente I

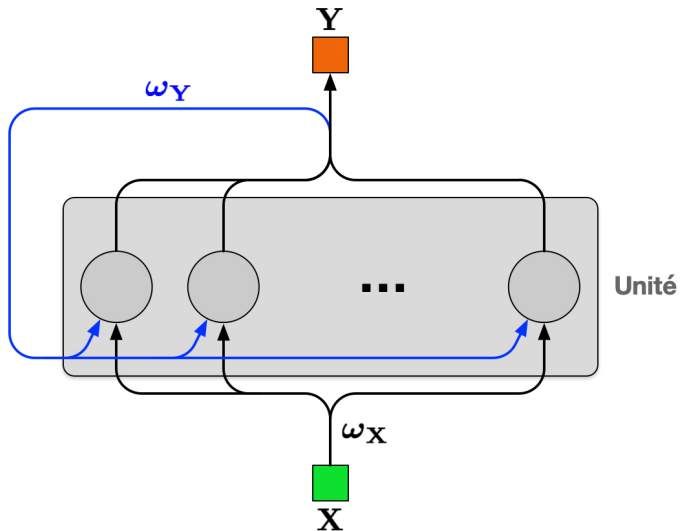


RNN

Autoencoders

Références

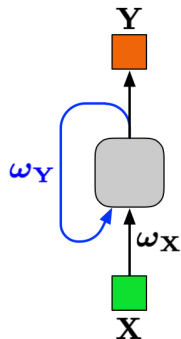
Unité récurrente II



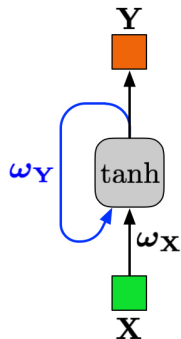
RNN

Autoencoders

Références



$$\mathbf{Y}_t = \varphi \left(\omega_{\mathbf{X}}^{\top} \mathbf{X}_t + \omega_{\mathbf{Y}}^{\top} \mathbf{Y}_{t-1} + \omega_0 \right)$$



$$\mathbf{Y}_t = \tanh(\omega_{\mathbf{X}}^{\top} \mathbf{X}_t + \omega_{\mathbf{Y}}^{\top} \mathbf{Y}_{t-1} + \omega_0)$$

Des difficultés et des palliatifs

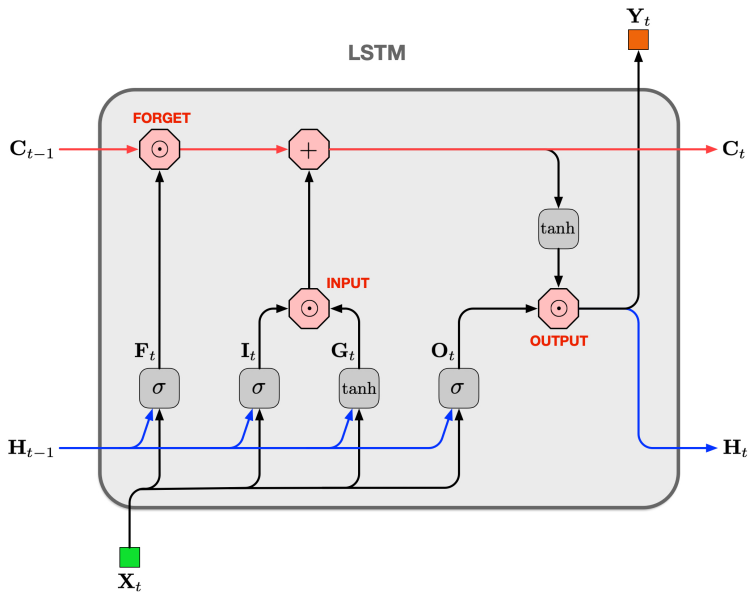
RNN

Autoencoders

Références

- ▶ Des difficultés :
 - ▶ Convergence lente.
 - ▶ Problème d'*exploding gradient* / *vanishing gradient*.
 - ▶ Mémoire très faible.
- ▶ Des palliatifs :
 - ▶ LSTM (Long Short Term Memory) :
(Hochreiter et Schmidhuber, 1997)
 - ▶ GRU (Gated Recurrent Unit) :
(Chung et collab., 2014)

LSTM



RNN

Autoencoders

Références

LSTM : notations

- ▶ $\mathbf{X} \in \mathbb{R}^p$: vecteur des entrées (*input vector*).
- ▶ $\mathbf{F} \in \mathbb{R}^h$: vecteur d'activation de la porte d'oubli (*forget gate's activation function*).
- ▶ $\mathbf{I} \in \mathbb{R}^h$: vecteur d'activation de la porte d'entrée (*input gate's activation function*).
- ▶ $\mathbf{O} \in \mathbb{R}^h$: vecteur d'activation de la porte de sortie (*output gate's activation function*).
- ▶ $\mathbf{C} \in \mathbb{R}^h$: vecteur d'état de l'unité (*cell state vector*).
- ▶ $\mathbf{H} \in \mathbb{R}^h$: vecteur d'état caché (*hidden state vector*).
- ▶ $\mathbf{Y} \in \mathbb{R}^h$: vecteur des sorties (*output vector*).
- ▶ $\omega_{\cdot, \cdot}$: matrices de poids.
- ▶ $\omega_{\cdot, 0}$: vecteurs de biais.
- ▶ $\odot$: produit de Hadamard.

Pour deux matrices de mêmes dimensions :

$$(\mathbf{A} \odot \mathbf{B})_{ij} = (\mathbf{A})_{ij} (\mathbf{B})_{ij} .$$

- ▶ σ : fonction d'activation sigmoïde.
- ▶ $\tanh$: fonction d'activation tangente hyperbolique.

$$\mathbf{F}_t = \sigma \left(\omega_{\mathbf{F},\mathbf{C}}^\top \mathbf{C}_{t-1} + \omega_{\mathbf{F},\mathbf{X}}^\top \mathbf{X}_t + \omega_{\mathbf{F},0} \right) ,$$

$$\mathbf{I}_t = \sigma \left(\omega_{\mathbf{I},\mathbf{C}}^\top \mathbf{C}_{t-1} + \omega_{\mathbf{I},\mathbf{X}}^\top \mathbf{X}_t + \omega_{\mathbf{I},0} \right) ,$$

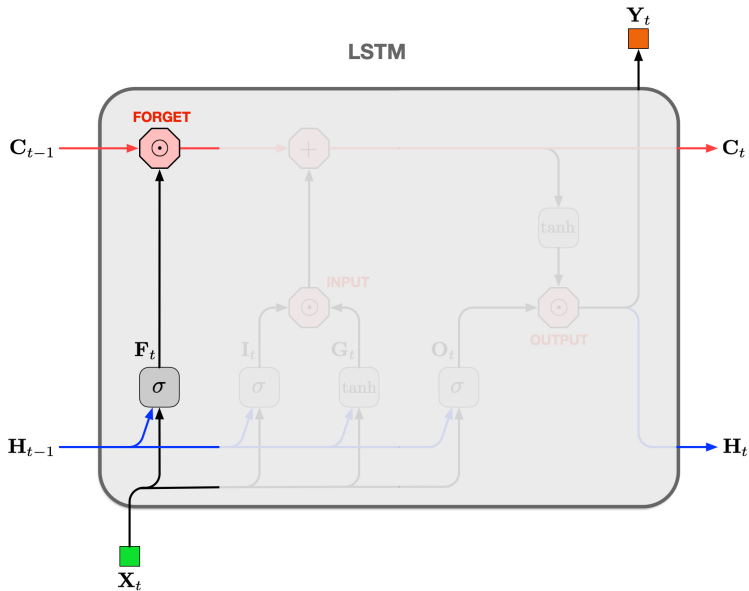
$$\mathbf{G}_t = \tanh \left(\omega_{\mathbf{G},\mathbf{C}}^\top \mathbf{C}_{t-1} + \omega_{\mathbf{G},\mathbf{X}}^\top \mathbf{X}_t + \omega_{\mathbf{G},0} \right) ,$$

$$\mathbf{O}_t = \sigma \left(\omega_{\mathbf{O},\mathbf{C}}^\top \mathbf{C}_{t-1} + \omega_{\mathbf{O},\mathbf{X}}^\top \mathbf{X}_t + \omega_{\mathbf{O},0} \right) ,$$

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \mathbf{G}_t ,$$

$$\mathbf{Y}_t = \mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) .$$

LSTM : porte d'oubli I



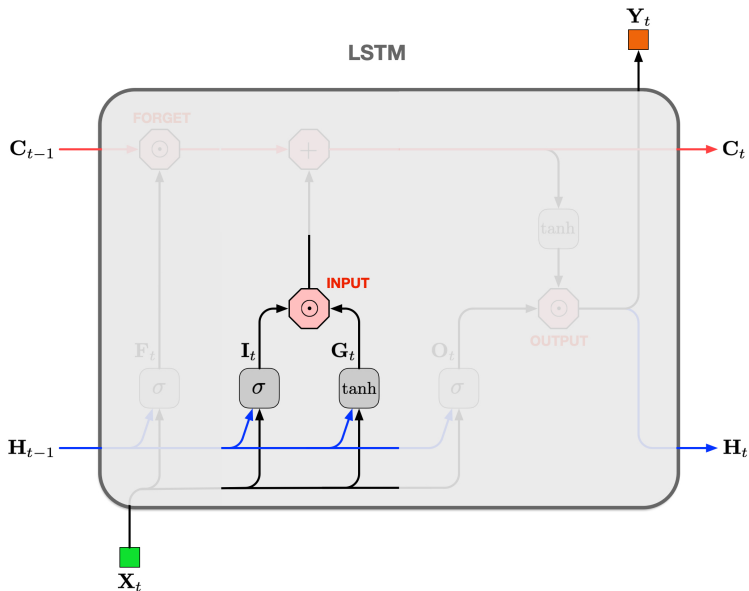
RNN

Autoencoders

Références

- ▶ La **porte d'oubli** (*forget gate*) a pour rôle de déterminer quelles informations doivent être conservées ou non.
- ▶ Pour cela, l'état caché précédent est concaténé à la donnée d'entrée, puis l'ensemble est soumis à la fonction sigmoïde.
- ▶ L'information sera « oubliée » si la valeur est proche de 0, a contrario mémorisée et transmise aux étapes suivantes du traitement si la valeur proche de 1.

LSTM : porte d'entrée I



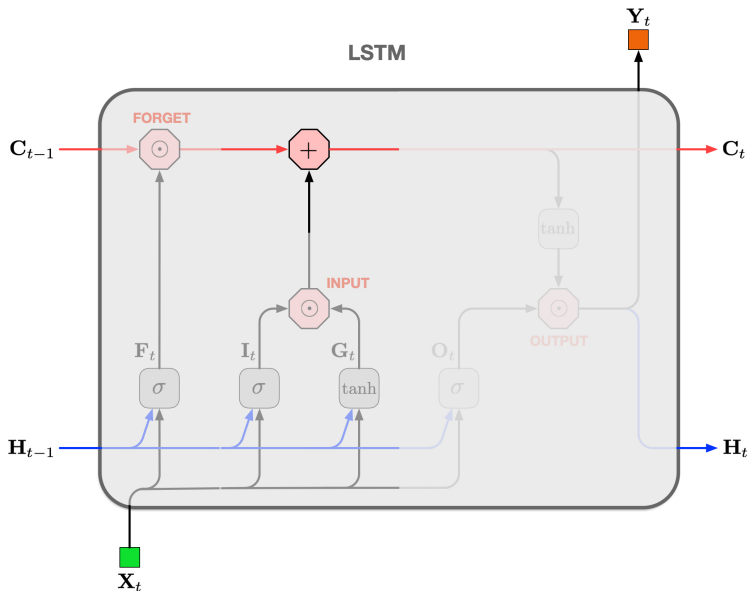
RNN

Autoencoders

Références

- ▶ La **porte d'entrée** (*input gate*) contrôle la mise à jour de la mémoire interne.
- ▶ Pour cela l'état caché précédent est également concaténé à l'entrée courante, puis l'ensemble est soumis à deux fonctions : la sigmoïde qui décide quelles composantes seront mises à jour, et la tangente hyperbolique qui génère les nouvelles valeurs candidates à insérer dans l'état de cellule.
- ▶ Ces deux résultats sont multipliés élément par élément, ce qui permet de filtrer finement l'information à intégrer dans la mémoire.

LSTM : mise à jour de l'état de l'unité l



RNN

Autoencoders

Références

LSTM : mise à jour de l'état de l'unité II

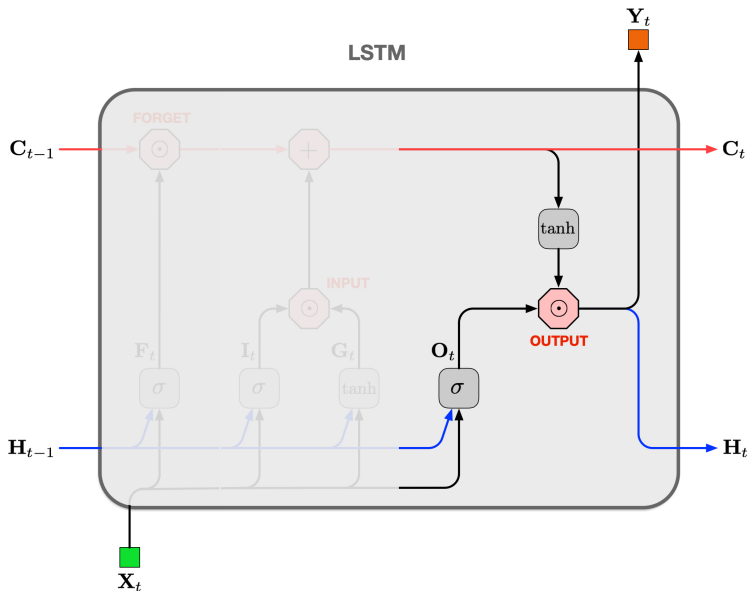
RNN

Autoencoders

Références

- ▶ L'**état de l'unité** (la mémoire longue) est mis à jour en combinant l'information conservée par la porte d'oubli et celle transmise par la porte d'entrée.
- ▶ En pratique, une partie de l'ancienne mémoire est « effacée » (pondérée par la sortie de la porte d'oubli), tandis qu'une autre est actualisée avec les nouvelles valeurs pertinentes, sélectionnées par la porte d'entrée.
- ▶ Ce mécanisme permet au réseau de conserver des informations sur le long terme tout en les ajustant dynamiquement.

LSTM : porte de sortie I



RNN

Autoencoders

Références

LSTM : porte de sortie II

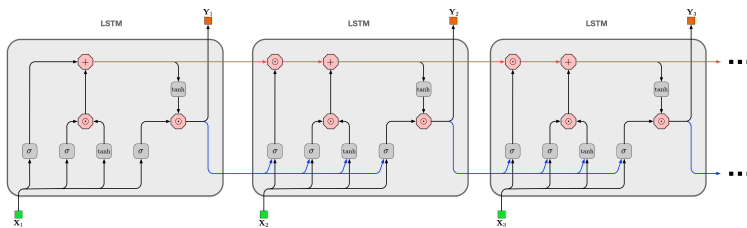
RNN

Autoencoders

Références

- ▶ La porte de sortie (*output gate*) détermine l'état caché actuel, c'est-à-dire ce que le réseau va transmettre à l'étape suivante.
- ▶ Elle combine à nouveau l'état caché précédent et l'entrée actuelle, sélectionne les composantes pertinentes via la fonction sigmoïde puis multiplie ce résultat par une version amendée de la mémoire interne (par la tangente hyperbolique).
- ▶ Le produit final constitue l'état caché transmis à la prochaine unité LSTM ou à la couche de sortie.

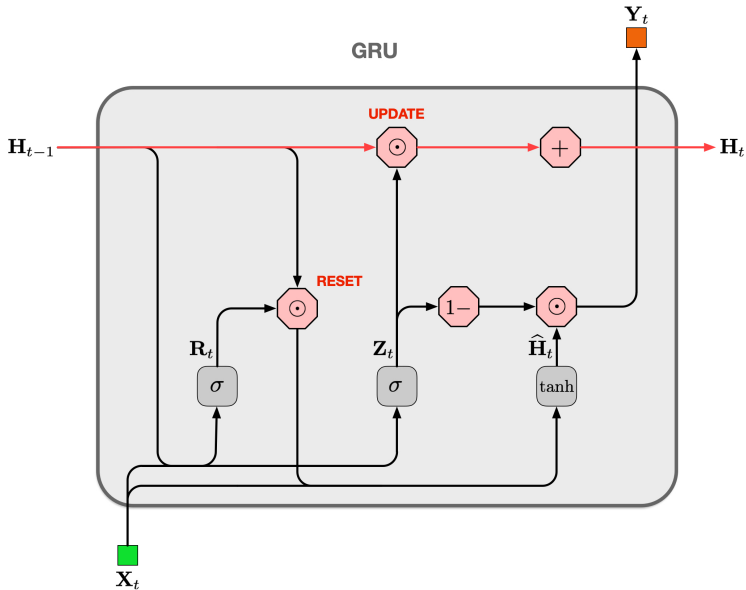
LSTM en chaîne



RNN

Autoencoders

Références



GRU : notations

- ▶ $\mathbf{X} \in \mathbb{R}^p$: vecteur des entrées (*input vector*).
- ▶ $\mathbf{H} \in \mathbb{R}^h$: vecteur d'état caché (*hidden state vector*).
- ▶ $\hat{\mathbf{H}} \in \mathbb{R}^h$: vecteur d'état caché (*candidate activation vector*).
- ▶ $\mathbf{Z} \in [0, 1]^h$: vecteur de porte mis à jour (*update gate vector*).
- ▶ $\mathbf{R} \in [0, 1]^h$: vecteur de porte réinitialisé (*reset gate vector*).
- ▶ $\mathbf{Y} \in \mathbb{R}^h$: vecteur des sorties (*output vector*).
- ▶ $\omega_{\cdot, \cdot}$: matrices de poids.
- ▶ $\omega_{\cdot, 0}$: vecteurs de biais.
- ▶ $\odot$: produit de Hadamard.
Pour deux matrices de mêmes dimensions :

$$(\mathbf{A} \odot \mathbf{B})_{ij} = (\mathbf{A})_{ij} (\mathbf{B})_{ij} .$$

- ▶ σ : fonction d'activation sigmoïde.
- ▶ $\tanh$: fonction d'activation tangente hyperbolique.

$$\mathbf{Z}_t = \sigma \left(\omega_{\mathbf{Z},\mathbf{X}}^\top \mathbf{X}_t + \omega_{\mathbf{Z},\mathbf{H}}^\top \mathbf{H}_{t-1} + \omega_{\mathbf{Z},0} \right) ,$$

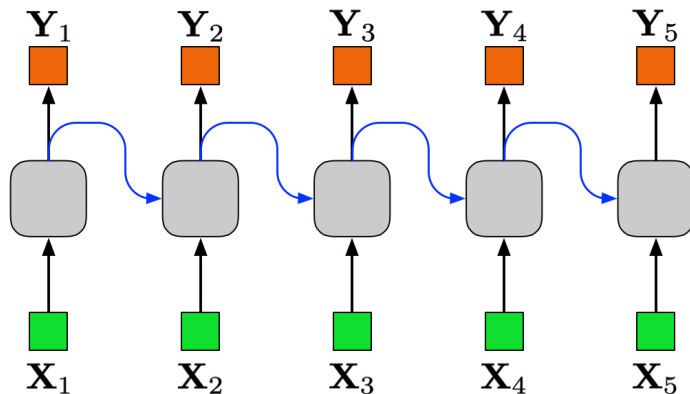
$$\mathbf{R}_t = \sigma \left(\omega_{\mathbf{R},\mathbf{X}}^\top \mathbf{X}_t + \omega_{\mathbf{R},\mathbf{H}}^\top \mathbf{H}_{t-1} + \omega_{\mathbf{R},0} \right) ,$$

$$\hat{\mathbf{H}}_t = \tanh \left(\omega_{\hat{\mathbf{H}},\mathbf{X}}^\top \mathbf{X}_t + \omega_{\hat{\mathbf{H}},\mathbf{RH}}^\top \mathbf{R}_t \odot \mathbf{H}_{t-1} + \omega_{\hat{\mathbf{H}},0} \right) ,$$

$$\mathbf{Y}_t = \mathbf{H}_t = (1 - \mathbf{Z}_t) \odot \mathbf{H}_{t-1} + \mathbf{Z}_t \odot \hat{\mathbf{H}}_t .$$

Application potentielle : *serie-to-serie*

Exemple : prévision de séries temporelles.



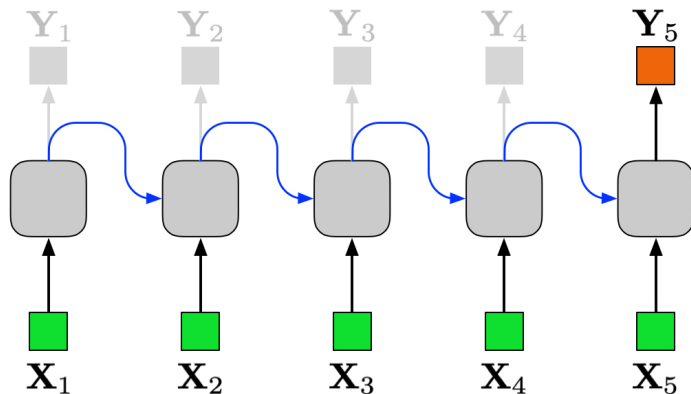
RNN

Autoencoders

Références

Application potentielle : *serie-to-vector*

Exemple : analyse de sentiment.



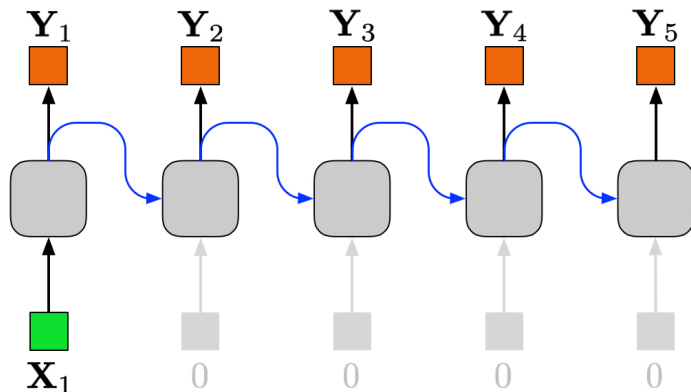
RNN

Autoencoders

Références

Application potentielle : *vector-to-serie*

Exemple : annotation d'images.



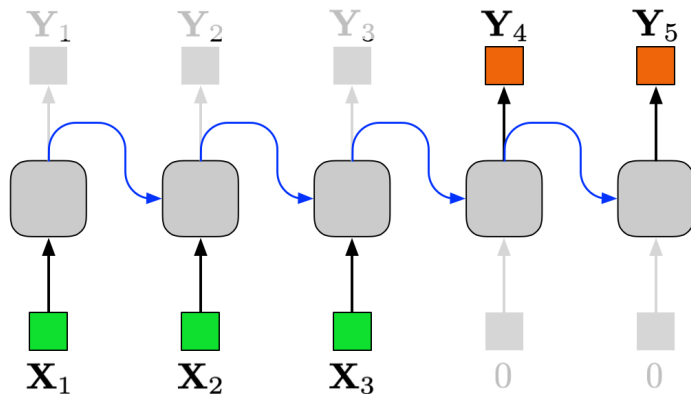
RNN

Autoencoders

Références

Application potentielle : *vector-to-vector*

Exemple : traduction.



RNN

Autoencoders

Références

Plan

RNN

Autoencoders

Références

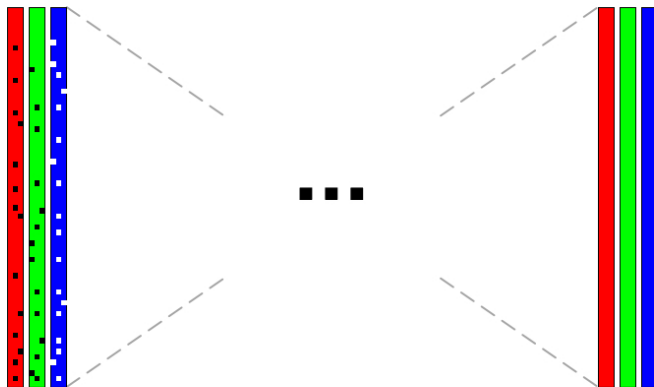
Autoencoders

Autoencoders I

RNN

Autoencoders

Références

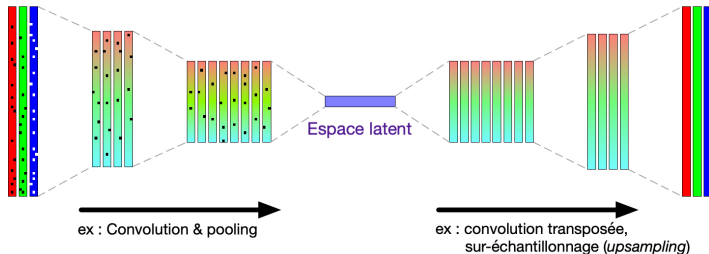


Autoencoders II

RNN

Autoencoders

Références



Autoencoders III

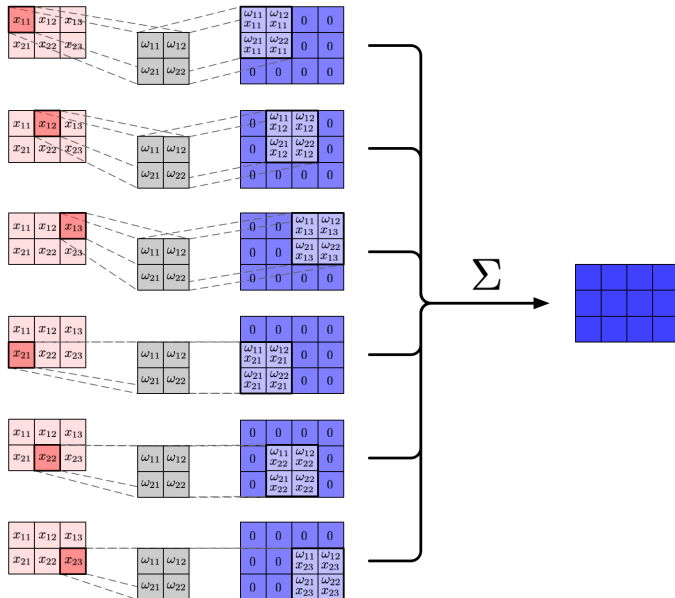
RNN

Autoencoders

Références



Convolution transposée I



RNN

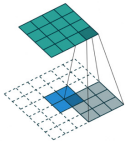
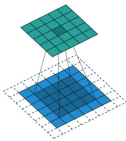
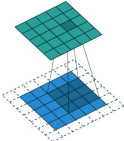
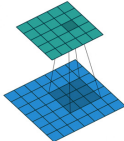
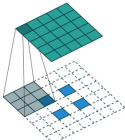
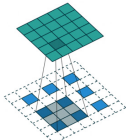
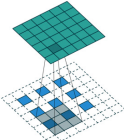
Autoencoders

Références

Convolution transposée II

Transposed convolution animations

N.B.: Blue maps are inputs, and cyan maps are outputs.

			
No padding, no strides, transposed	Arbitrary padding, no strides, transposed	Half padding, no strides, transposed	Full padding, no strides, transposed
			
No padding, strides, transposed	Padding, strides, transposed	Padding, strides, transposed (odd)	

https://github.com/vdumoulin/conv_arithmetic

RNN

Autoencoders

Références

Convolution

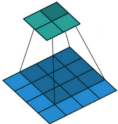
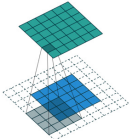
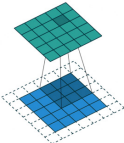
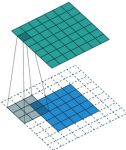
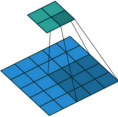
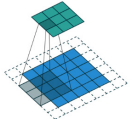
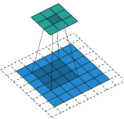
RNN

Autoencoders

Références

Convolution animations

N.B.: Blue maps are inputs, and cyan maps are outputs.

			
No padding, no strides	Arbitrary padding, no strides	Half padding, no strides	Full padding, no strides
			
No padding, strides	Padding, strides	Padding, strides (odd)	

https://github.com/vdumoulin/conv_arithmetic

Upsampling

x_{11}	x_{12}	x_{13}
x_{21}	x_{22}	x_{23}

Upsampling 2D
→

x_{11}	x_{11}	x_{12}	x_{12}	x_{13}	x_{13}
x_{11}	x_{11}	x_{12}	x_{12}	x_{13}	x_{13}
x_{21}	x_{21}	x_{22}	x_{22}	x_{23}	x_{23}
x_{21}	x_{21}	x_{22}	x_{22}	x_{23}	x_{23}

RNN

Autoencoders

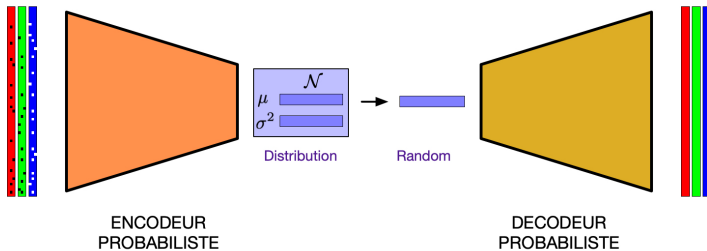
Références

Variational autoencoder

RNN

Autoencoders

Références



- Arias, S., E. Maldonado et J.-L. Parouty. 2024, «Fidle», URL <https://fidle.cnrs.fr>.
- Chollet, F. 2020, *L'apprentissage profond avec Python*, Machinelearning.fr.
- Chollet, F. 2021, *Deep learning with Python*, 2^e éd., Manning.
- Chung, J., C. Gulcehre, K. Cho et Y. Bengio. 2014, «Empirical evaluation of gated recurrent neural networks on sequence modeling», *arXiv preprint arXiv :1412.3555*.
- Dumoulin, V. et F. Visin. 2018, «A guide to convolution arithmetic for deep learning», *arXiv preprint arXiv :1603.07285*.
- Hochreiter, S. et J. Schmidhuber. 1997, «Long short-term memory», *Neural Computation*, vol. 9, n° 8, p. 1735–1780.